

Fase 3 – Projeto 5

ConnectHub: API de Gestão Colaborativa

O desafio que muda o seu nível

Este é o projeto que marca uma virada definitiva na sua formação.

Até aqui, você já construiu interfaces, trabalhou lógica de programação e fez aplicações reagirem no navegador. Agora, o desafio é maior: você vai construir também o que acontece **por trás da tela**. O **ConnectHub** é o projeto que separa quem sabe montar uma interface de quem realmente consegue entregar uma solução **full-stack**.

Aqui, você não vai apenas mostrar dados. Você vai criar a infraestrutura que permite que eles existam com segurança, persistam no tempo e sejam acessados por diferentes usuários em diferentes dispositivos.

Seu objetivo será desenvolver um sistema full-stack com **Node.js**, **banco de dados relacional** e **autenticação de usuários**, demonstrando que você entende o ciclo completo de uma requisição web: da interação na interface até o armazenamento seguro em um servidor real.

Posicionamento do projeto

O ConnectHub é um projeto de nível **avançado**, com foco em **backend**, **banco de dados** e **integração full-stack**, duração sugerida de **12 semanas** e execução **individual**, com possibilidade de simulação de revisão por pares ou trabalho em dupla para exercitar colaboração via Git.

Dentro da progressão do programa, este projeto encerra o ciclo de fundamentos. Ele exige que você conecte tudo o que aprendeu antes:

- a construção de telas da **Fase 1**
- a lógica dinâmica da **Fase 2**
- e agora a robustez arquitetural da **Fase 3**

É nesta fase que o projeto deixa de parecer apenas um exercício e começa a se comportar como um **produto viável**.

O cenário que justifica a solução

Imagine que a aplicação criada antes, como o SmartCash, agora precisa ser usada não por uma única pessoa, mas por uma **família** ou por uma **pequena empresa**.

Se os dados continuarem presos ao navegador de apenas um usuário, via **LocalStorage**, a colaboração se torna impossível. Basta trocar de dispositivo, limpar o cache ou abrir em outro computador para tudo desaparecer. Além disso, não existe segurança real: quem tiver acesso à máquina poderá visualizar os dados.

É exatamente dessa limitação que nasce o ConnectHub.

A dor aqui é clara: aplicações isoladas não sustentam uso compartilhado, sincronização nem privacidade. O mercado exige sistemas em que o usuário possa fazer login de qualquer lugar, encontrar suas informações intactas e confiar que seus dados estão protegidos.

Seu papel será construir esse hub central: um ambiente em que os dados deixam de morar apenas no navegador e passam a ser processados por um servidor, armazenados em um banco relacional e protegidos por autenticação e controle de acesso.

Quem está envolvido

Este projeto simula um cenário em que você assume mais de um papel dentro do desenvolvimento.

Como **desenvolvedor backend**, você será responsável por criar as portas de entrada do sistema: os **endpoints da API**, as regras de negócio e a persistência correta no banco de dados.

Como **desenvolvedor frontend**, você será responsável por conectar a interface construída nas fases anteriores a esse novo cérebro na nuvem.

O **usuário final** precisa de algo simples de entender, mas tecnicamente confiável: uma conta com login e senha, acesso seguro aos próprios dados e sincronização entre múltiplos dispositivos.

Objetivo do projeto

Ao final das 12 semanas, você deverá entregar um **sistema completo**, composto por:

- um **servidor backend em Node.js**
- um **banco de dados relacional SQL**
- uma **interface front-end** integrada a esse backend

O objetivo é transformar a aplicação em um sistema capaz de:

- salvar dados de forma permanente em um servidor
- autenticar usuários
- proteger informações sensíveis
- permitir operações remotas de criação, leitura, atualização e exclusão de dados

O foco principal está na robustez do backend. A interface não precisa ser visualmente complexa, mas precisa se integrar de forma funcional à API, mostrando que a tela e o servidor realmente conversam entre si.

A visão de sucesso é clara: o usuário consegue se cadastrar no celular, registrar dados e reencontrá-los exatamente iguais ao abrir o sistema em outro dispositivo, com segurança e fluidez.

O que sua solução precisa entregar

Sua entrega principal será um sistema full-stack funcional.

Ele deverá incluir:

- **sistema de autenticação**, com criação de contas e login seguro
- **persistência em banco de dados relacional**, substituindo de vez o LocalStorage
- **operações CRUD via API**, permitindo criar, ler, atualizar e deletar dados remotamente
- **segurança de dados**, com criptografia de senhas e controle de acesso por **tokens JWT**

O que não é o foco principal

Neste projeto, o centro da avaliação não está em criar uma interface visual sofisticada.

A prioridade é garantir:

- backend robusto
- armazenamento permanente
- segurança
- validação
- integração funcional entre frontend e servidor

Como o projeto evolui ao longo dos 3 meses

O desenvolvimento está dividido em quatro blocos de três semanas. Cada etapa adiciona uma camada essencial da arquitetura full-stack.

Marco 1: Fundação e Rotas

Semanas 1 a 3

Você começa pela base do servidor.

O entregável desta etapa é um servidor **Node.js** em funcionamento, com rotas básicas do tipo **Hello World** e uma estrutura de pastas organizada. Também entra aqui a criação do primeiro arquivo de rotas, com um CRUD inicial em memória.

O critério de transição é objetivo: você precisa conseguir enviar uma requisição e receber uma resposta **JSON** simulada, seja no navegador, seja em uma ferramenta de teste.

Marco 2: O Coração dos Dados

Semanas 4 a 6

Com a base do servidor pronta, você traz o banco para o centro da arquitetura.

O entregável desta fase é a integração com um banco de dados **SQL**, como **MySQL** ou **PostgreSQL**, incluindo a criação das tabelas de **usuário** e **transações/dados**.

O critério de transição é que os dados enviados pela API passem a ser gravados permanentemente no banco e não desapareçam ao reiniciar o servidor.

Marco 3: Muralha de Segurança

Semanas 7 a 9

Agora, a prioridade é proteger o sistema.

O entregável será a implementação de **cadastro** e **login**, com uso de **criptografia de senhas** e geração de **tokens JWT** para manter o usuário autenticado.

O critério de transição é direto: se alguém tentar acessar dados de outro usuário sem o token correto, a API deve responder com erro de **Não Autorizado**.

Marco 4: A Grande Integração

Semanas 10 a 12

Na etapa final, você conecta todas as partes.

O entregável é a integração da tela com a API, substituindo o LocalStorage por chamadas reais ao servidor, além dos testes finais e do deploy.

O checkpoint final exige que o sistema completo esteja rodando na nuvem, acessível por celular e computador, com login funcional e dados protegidos.

Conceitos fundamentais que você precisa dominar

Para construir o ConnectHub, você precisa sair da lógica de interface e entrar no território da arquitetura, da segurança e da comunicação entre camadas.

Protocolo HTTP e verbos REST

Você deverá compreender como navegador e servidor se comunicam por meio de requisições como **GET**, **POST**, **PUT** e **DELETE**, além de interpretar corretamente **status codes** como **200 OK**, **404 Not Found** e **500 Internal Server Error**.

Ambiente runtime com Node.js

Será necessário entender como o JavaScript é executado no servidor, como os pacotes são gerenciados com **NPM** e como organizar o projeto em módulos.

Banco de dados relacional e SQL

Você deverá dominar tabelas, tipos de dados, chaves primárias (**PK**), chaves estrangeiras (**FK**) e consultas para salvar e recuperar informações com consistência.

Modelagem de dados

Será essencial desenhar corretamente as relações entre entidades, como no caso em que **um usuário possui muitas transações**.

Autenticação e autorização

Você precisará dominar a diferença entre:

- **autenticação**: identificar quem é o usuário
- **autorização**: definir o que ele pode acessar ou fazer

Criptografia básica

Senhas jamais devem ser salvas em texto puro. Você precisará aplicar conceitos de **hashing** para proteger credenciais.

Assincronismo avançado

Consultas ao banco e chamadas externas exigem domínio de **Promises** e **async/await**, para que a aplicação continue fluida mesmo enquanto aguarda operações de I/O.

Estratégia de estudo com IA

Neste projeto, a IA deve ajudar você a desmontar a complexidade do backend em blocos menores e mais compreensíveis.

A prioridade recomendada é:

- **60% do tempo em Node.js e SQL**
- **40% do tempo em integração frontend-backend**

O caminho mais inteligente não é tentar aprender tudo de uma vez. Construa uma rota, teste. Depois, conecte ao banco. Só então avance para segurança e autenticação.

Alguns prompts úteis para esse processo são:

- “Vou criar um sistema de gestão. Preciso de uma tabela de usuários e uma de transações. Explique como criar uma chave estrangeira (FK) para que cada transação pertença a um usuário específico no SQL.”
- “Meu servidor Node.js retornou um erro ‘EADDRINUSE’. O que isso significa e como posso liberar a porta no meu terminal?”
- “Atue como um especialista em segurança. Analise este meu código de login [colar código] e verifique se estou protegendo a senha do usuário corretamente antes de salvar no banco.”

Você também pode alinhar teoria e milestone:

- Para o **Marco 1**, peça uma explicação do padrão **MVC**
- Para o **Marco 2**, use a IA para gerar queries de teste e validar a estrutura do banco
- Para o **Marco 3**, peça um passo a passo do fluxo completo do **JWT**, do login ao acesso protegido

Para fortalecer a camada de defesa da aplicação, use este protocolo:

“Simule uma requisição maliciosa para a minha API de cadastro e me ajude a criar validações que impeçam dados corrompidos de entrar no meu banco.”

O que você deve aprender ao concluir este projeto

Ao final do ConnectHub, você deverá ser capaz de:

- arquitetar e construir uma **API RESTful robusta** com **Node.js**
- modelar e gerenciar **bancos de dados relacionais**
- implementar **segurança de nível profissional**, com hashing de senhas e tokens JWT
- integrar o ciclo completo **frontend + backend + banco**
- dominar **assincronismo** com **Promises** e **async/await**
- validar dados e tratar erros antes que eles comprometam o banco
- publicar a API e o banco em produção, tornando o sistema acessível de qualquer lugar

Requisitos e restrições

Esta fase possui obrigações técnicas claras. O objetivo é garantir que você construa uma API com padrão de mercado e respeito à privacidade dos dados.

Requisitos funcionais

O sistema deve:

- permitir o **cadastro de novos usuários**
- permitir a **autenticação** dos usuários existentes
- garantir que um usuário autenticado só consiga **visualizar, criar, editar ou excluir os próprios dados**

- armazenar todas as informações em um **banco de dados relacional SQL**
- permitir que o frontend consuma os dados da API em tempo real, refletindo na interface as mudanças feitas no servidor

Requisitos não funcionais

- **Proteção de credenciais:** é proibido salvar senhas em texto puro; deve-se usar hashing com algoritmos como **BCrypt**
- **Sessões seguras:** o acesso às rotas privadas deve ser controlado por **JWT**
- **Tratamento de exceções:** a API deve retornar códigos HTTP apropriados, como **401** para não autorizado e **404** para não encontrado

Restrições técnicas

- O backend deve ser desenvolvido **obrigatoriamente em Node.js**
- O banco deve ser **relacional**, utilizando **MySQL**, **PostgreSQL** ou **SQLite**
- O uso de bancos **NoSQL**, como **MongoDB**, está restrito nesta fase
- A API deve seguir os princípios **REST**, utilizando corretamente **GET**, **POST**, **PUT** e **DELETE**

Restrições de conformidade

- O código de frontend e backend deve estar **bem organizado**, preferencialmente em repositórios separados ou em uma estrutura de pastas clara
- O projeto deve conter **instruções de instalação no README**

Recursos e documentação para construir com padrão profissional

Esta fase exige ferramentas de teste, diagnóstico e validação mais próximas do que o mercado realmente utiliza.

Ferramentas de teste de API

- **Postman** ou **Insomnia**, para criar requisições, testar JSON e validar o comportamento dos tokens

Gestão de banco de dados

- **DBeaver** ou **MySQL Workbench**, para visualizar tabelas, relacionamentos e executar comandos SQL diretamente no banco

Bibliotecas e frameworks Node.js

- **Express.js**, para criar rotas e gerenciar o servidor
- **Bcrypt**, para criptografia de senhas
- **JSON Web Token (JWT)**, para geração e validação de tokens

Documentação técnica de referência

- **Documentação do Express**, para roteamento e middlewares
- **MDN – HTTP Status Codes**, para saber qual código retornar em cada situação
- **W3Schools SQL Tutorial**, como guia rápido para inserção, seleção e junção de tabelas

Monitoramento de erros

- **Nodemon**, para reiniciar o servidor automaticamente a cada alteração no código

Riscos e pontos de atenção

Nesta fase, os riscos se tornam mais sensíveis porque qualquer erro pode comprometer segurança, integridade ou disponibilidade da aplicação.

Exposição de credenciais

Você nunca deve enviar senhas, chaves de API ou segredos diretamente no código. O uso de **variáveis de ambiente (.env)** é obrigatório para evitar vazamento de dados sensíveis no GitHub.

Injeção de SQL

Concatenar variáveis diretamente em queries é um risco grave. Um usuário mal-intencionado pode enviar comandos capazes de comprometer ou apagar dados do banco.

Promessas não tratadas

Se o banco falhar e o seu código não tiver **try/catch**, o backend pode cair completamente, deixando o usuário preso em uma tela de carregamento sem resposta.

Conflito de CORS

Durante a integração, é comum que o navegador bloqueie a comunicação entre frontend e backend por estarem em endereços diferentes. Esse é um dos primeiros pontos críticos da etapa de integração.

Dados órfãos

Ao deletar um usuário, você precisa definir o que acontecerá com os registros ligados a ele. Sem uma lógica adequada, o banco passa a acumular dados sem dono.

Logs de erro são parte do trabalho

Nesta fase, você precisa aprender a ler não apenas o console do navegador, mas também o terminal do backend. Muitas vezes, a causa real do problema aparece apenas ali.

Testes de stress simples

Também vale observar o comportamento do sistema em carga básica: o que acontece se muitas requisições forem feitas em sequência? A aplicação continua estável?

Critérios de aceite

O ConnectHub só pode ser considerado concluído quando segurança, integração e persistência estiverem efetivamente funcionando.

Checklist de funcionalidades

- O usuário consegue criar uma conta e realizar login com sucesso
- A aplicação impede o acesso a páginas ou dados privados quando o usuário não está autenticado
- O CRUD completo está funcionando e os dados persistem no banco SQL
- O frontend exibe mensagens de erro claras, como “Senha incorreta” ou “Erro de conexão com o servidor”

Qualidade técnica e segurança

- As senhas dos usuários estão criptografadas no banco por meio de hash
- O backend utiliza **JWT** para validar sessões e proteger rotas
- Todas as variáveis sensíveis estão em arquivo **.env** e não foram enviadas ao repositório público
- O código segue estrutura de pastas organizada, como **MVC** ou similar

Integração e experiência de uso

- A aplicação trata estado de **carregamento** enquanto aguarda respostas do servidor
- O sistema é totalmente responsivo e funciona tanto no celular quanto no computador

Critérios de entrega profissional

- O código-fonte do frontend e do backend está disponível no GitHub com **README detalhado**
- A API e o banco de dados foram publicados em nuvem e o link está funcional para testes externos

Possibilidades de expansão

Depois de concluir o escopo obrigatório, você pode levar o ConnectHub para uma camada mais próxima de produto escalável.

Comunicação em tempo real

Adicionar **WebSockets** com bibliotecas como **Socket.io** para atualizar a tela sem recarregar a página.

Upload e gestão de arquivos

Permitir envio de fotos de perfil ou anexos, aprendendo a lidar com armazenamento no servidor ou em nuvem, como **AWS S3**.

Autenticação social

Implementar login com **Google**, **GitHub** ou **LinkedIn**, usando **OAuth2**.

Integração de pagamentos

Simular ou implementar uma régua de cobrança com ferramentas como **Stripe** ou **Pagar.me**, aproximando a aplicação de um modelo SaaS.

Testes automatizados

Adicionar testes unitários e de integração com ferramentas como **Jest** ou **Supertest**, aumentando a confiabilidade do backend.

Cache e performance

Introduzir soluções como **Redis** para acelerar consultas recorrentes e reduzir carga no banco principal.

Versão mobile nativa

Aproveitar a API criada para desenvolver uma versão mobile com **React Native** ou **Flutter**.

Fechamento

O ConnectHub é o projeto em que você deixa de construir apenas funcionalidades isoladas e passa a arquitetar um sistema completo, seguro e persistente.

Ao concluir esse desafio, você terá demonstrado algo decisivo: que sabe fazer a informação sair da tela, atravessar a rede, ser validada, protegida, armazenada e devolvida ao usuário com consistência.

Esse é o momento em que seu portfólio começa a comunicar maturidade técnica de verdade.